KKR ERP

How to publish an update, step by step

This guide explains how a new version of the desktop app gets onto GitHub, and how a person already using the app installs that version.

The app does not update itself when it opens. A person opens Settings, then Updates, then presses the buttons. Those buttons download the latest published GitHub release.

The short version

- You build the app on your computer.
- You put the build files on a GitHub release.
- The person using the app presses Update to latest.

> **Remember**
> The first copy of this new updater must be installed by hand. After that, later versions can be installed from inside the app.

Part 1. Do this once

You only repeat this part if you change the GitHub account, the private signing key, or the computer you build on.

1

Open the .env file at the top of the project

This file is not uploaded to GitHub. It tells the build which GitHub repository will hold the updates.

- UPDATE_GITHUB_REPO is the repository, written as owner/name. Example: Ragworx/KKR
- UPDATE_GITHUB_TAG_PREFIX is the start of the release name. It is v, so version 1.0.1 becomes the tag v1.0.1.
- UPDATE_MANIFEST_NAME is the update list file. Leave it as latest.json.

Save the file. The app only learns a new repository after you build again.

2

Keep the signing key safe

The private key is the file frontend/src-tauri/.tauri/kkr-erp.key. The build uses it to prove an update really came from you.

- Copy this file to a safe place, such as a password manager or a private backup.
- Do not email it, and do not put it on GitHub.
- If this file is lost, installed copies of the app will refuse future updates.

3

Make sure the GitHub repository can be downloaded without logging in

The app on someone else's computer is not logged into GitHub. If the repository is private, the update button cannot see the release.

Open the release link in a private browser window later in this guide. If GitHub asks you to sign in, the repository is still private.

Part 2. Every time you want a new version

Do these steps in order. Use a higher version number than the one people already have installed.

1 Raise the version number

Open frontend/src-tauri/tauri.conf.json. Change the version. Example: change 1.0.0 to 1.0.1. Save the file.

2 Build the app

On a Mac, open Terminal in the project folder and run:

```
./scripts/macos/build_release.sh
```

On Windows, open PowerShell in the project folder and run:

```
./scripts/windows/build_release.ps1
```

Wait until the script says the build is finished. If it says the signing key is missing, go back to Part 1, step 2.

3 Find the files the script created

On a Mac they are in dist/macos/latest. On Windows they are in dist/windows/latest.

- latest.json is the list the app reads. It says which file to download.
- On a Mac, also take the file whose name ends in .app.tar.gz. This is the update itself. The .dmg file is only for a first-time install.
- On Windows, also take the file whose name ends in -setup.exe. This is both the update and the first-time installer.

4 If you built both Mac and Windows, combine the two lists

Each computer writes its own latest.json. You must upload one latest.json that contains both computers.

- Open both latest.json files in a text editor.
- Copy the Mac block and the Windows block into one platforms section.
- The Mac block is named darwin-aarch64 or darwin-x86_64. The Windows block is named windows-x86_64.
- The version at the top must be the new version, such as 1.0.1.
- Save that one combined file. That is the file you upload.

Part 3. Put the files on GitHub

1

Create the release

In a browser, open your repository on GitHub. Click Releases, then Draft a new release.

- Tag: v plus the version. For version 1.0.1, the tag is v1.0.1. This must match tauri.conf.json and the prefix in .env.
- Title: KKR ERP 1.0.1, using your real version number.
- Leave Set as a pre-release turned off.
- Leave Set as a draft turned off when you publish. A draft is invisible to the app.

2

Attach the files

Drag these files onto the release:

- The one latest.json file.
- The Mac .app.tar.gz file, if you built a Mac version.
- The Windows -setup.exe file, if you built a Windows version.
- The .dmg file as well, if someone still needs to install the app for the first time on a Mac.

The names of the uploaded files must be exactly the names written inside latest.json. Do not rename them.

3

Publish the release

Click Publish release. Do not leave it as a draft.

4

Check that a stranger can download it

Open a private browser window, where you are not signed in to GitHub. Go to this address, using your own repository name:

```
https://github.com/OWNER/REPO/releases/latest/download/latest.json
```

The file should download. If GitHub asks for a login, the app will not be able to update. Make the repository public, or make the release files publicly downloadable, then try again.

Part 4. What the person using the app does

1. **Install the updater once by hand**
   Anyone who still has an old copy, from before this update button existed, must install the new .dmg or Windows setup file one time. After that install, the Updates tab exists.

2. **Open Settings, then Updates**
   The page shows the version that is installed right now.

3. **Press Check for updates**
   The app looks at the latest GitHub release. If that release has a higher version number, the page says a new version is available.

If the installed version and the GitHub version are the same, the page says Up to date. That is normal. Raise the version and publish again when you actually have a newer build.

4. **Press Update to latest**
   The app downloads the signed file, installs it, and restarts. On Windows, an installer window may also appear. Leave it to finish.

When the button does not find an update

- The GitHub release is still a draft.
- The tag is not exactly v plus the version in tauri.conf.json.
- latest.json was not attached, or the file names inside it do not match the attached files.
- The repository is private, so the download asks for a login.
- The installed app was built before you changed UPDATE_GITHUB_REPO. Build and install once more so the app looks at the new repository.
- The version on GitHub is not higher than the version already installed.

> **Remember**
> Change the repository in .env, build again, then install that build once. Only that new build will look at the new GitHub repository.